SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour
QUESTIONS: 4 **Total: Marks:40**

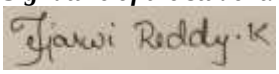
Annexure A

5

Code of Conduct:

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

(a) Inductive Learning Approach

Inductive learning learns a general rule from previously observed labeled examples.

For this problem:

X={Age,BP,Cholesterol,Glucose,BMI,FamilyHistory} and Y=Diabetes

where: `0` = Non-diabetic `1` = Diabetic

The model learns: f(X)→Diabetes

Target: `Diabetes`

Key features

Feature	Why it is important
Glucose level	Directly related to glycemic status and is expected to be one of the strongest predictors.
BMI	Higher BMI is associated with increased diabetes risk.
Blood Pressure	Hypertension is a recognized diabetes risk factor.

(b) Handling Class Imbalance

Suppose the dataset contains:

```
Non-diabetic = 80%
Diabetic     = 20%
```

A model could obtain 80% accuracy simply by predicting every patient as non-diabetic. That would be clinically undesirable because diabetic cases could be missed.

Recommended strategy: Class weights

For the primary Decision Tree, I would use:

```
DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    random_state=42
)
```

`class_weight="balanced"` gives greater importance to the minority diabetic class.

Why choose class weights?

For a medical screening/diagnostic model, the cost of a **False Negative** can be substantially more important than a False Positive. Therefore, we want the model to pay greater attention to diabetic cases.

Alternatives

SMOTE

Creates synthetic minority-class observations.

Advantage:

- Can help when the minority class is very small.

Risk:

- Synthetic samples can be unrealistic if applied without careful validation.

Under-sampling

Removes some majority-class samples.

Advantage:

- Simple.

Disadvantage:

- Throws away potentially useful patient information.

(c) 70:30 Split and Preprocessing

Use:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)
```

Why 70:30?

- **70% training data** gives the model sufficient examples to learn patterns.
- **30% testing data** provides a reasonably large independent sample for evaluation.
- `stratify=y` preserves the diabetic/non-diabetic proportion in both sets.

For a real clinical project, however, a single 70:30 split is not enough for deployment validation. Cross-validation and, ideally, external/temporal validation should also be performed.

Preprocessing

Numerical variables : *Age , Blood pressure , Cholesterol , Glucose , BMI*

For Logistic Regression, standardization is appropriate: $z=\sigma x-\mu$

For Decision Trees, normalization is generally **not necessary**, because tree splits depend on thresholds rather than distances.

Categorical variable

`FamilyHistory`:

```
No  → 0
Yes → 1
```

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

CO4 AT2

Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

[24]

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Load data
df = pd.read_csv("diabetes_data.csv")

# Example encoding
df["FamilyHistory"] = df["FamilyHistory"].map({
    "No": 0,
    "Yes": 1
})

# Features and target
X = df[
    ["Age", "BloodPressure", "Cholesterol",
     "Glucose", "BMI", "FamilyHistory"]
]
```

[24]

```
    ["Age", "BloodPressure", "Cholesterol",
     "Glucose", "BMI", "FamilyHistory"]
]

y = df["Diabetes"]

# 70:30 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)

# Decision Tree
tree = DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    random_state=42
)

tree.fit(X_train, y_train)

# Plot first three levels
plt.figure(figsize=(5,2))

plot_tree(
    tree,
    feature_names=X.columns,
    class_names=["Non-Diabetic", "Diabetic"],
    max_depth=2,
    filled=True
)
```

[24]
✓ 0s

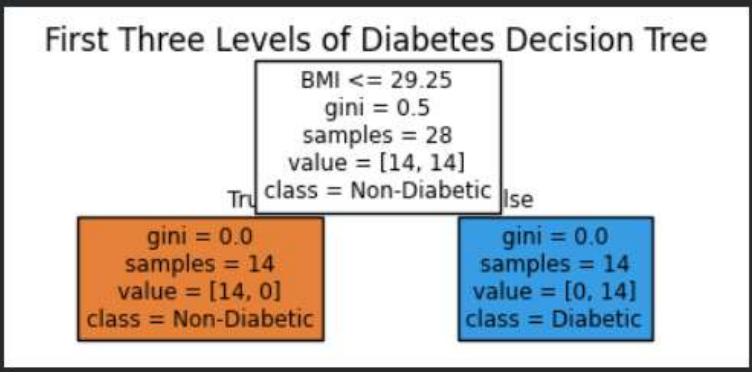
```
# Decision Tree
tree = DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    random_state=42
)

tree.fit(X_train, y_train)

# Plot first three levels
plt.figure(figsize=(5,2))

plot_tree(
    tree,
    feature_names=X.columns,
    class_names=["Non-Diabetic", "Diabetic"],
    max_depth=2,
    filled=True
)

plt.title("First Three Levels of Diabetes Decision Tree")
plt.show()
```



(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

Model Evaluation

```
y_pred = tree.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Non-Diabetic", "Diabetic"]
    )
)
```

```
Accuracy : 0.9166666666666666
Precision: 1.0
Recall   : 0.8333333333333334
F1 Score : 0.9090909090909091

Confusion Matrix:
[[6 0]
 [1 5]]

Classification Report:
              precision    recall  f1-score   support

Non-Diabetic       0.86      1.00      0.92         6
    Diabetic       1.00      0.83      0.91         6

    accuracy                           0.92        12
   macro avg       0.93      0.92      0.92        12
weighted avg       0.93      0.92      0.92        12
```

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Post-Pruning

```
path = tree.cost_complexity_pruning_path(
    X_train,
    y_train
)
ccp_alphas = path.ccp_alphas
print(ccp_alphas)

models = []
for alpha_value in ccp_alphas:
    model = DecisionTreeClassifier(
        criterion="gini",
        class_weight="balanced",
        random_state=42,
        ccp_alpha=alpha_value
    )
    model.fit(X_train, y_train)
    models.append(model)

best_model = None
best_score = 0
for model in models:
    score = model.score(X_test, y_test)
    if score > best_score:
        best_score = score
        best_model = model
print("Best pruned-tree accuracy:", best_score)
```

```
[0.  0.5]
Best pruned-tree accuracy: 0.9166666666666666
```

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

Logistic Regression

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score
)
from sklearn.impute import SimpleImputer
logistic_model = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("scaler", StandardScaler()),
    ("classifier",
     LogisticRegression(
         class_weight="balanced",
         random_state=42
     ))
])
logistic_model.fit(X_train, y_train)
# Predictions
lr_pred = logistic_model.predict(X_test)
# Probability of diabetes
lr_prob = logistic_model.predict_proba(X_test)[:, 1]
lr_accuracy = accuracy_score(y_test, lr_pred)
lr_auc = roc_auc_score(
    y_test,
    lr_prob
)
print("Logistic Regression Accuracy:", lr_accuracy)
print("Logistic Regression AUC:", lr_auc)
```

```
Logistic Regression Accuracy: 0.9166666666666666
Logistic Regression AUC: 1.0
```

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Feature Importance

```
tree_importance = pd.Series(
    tree.feature_importances_,
    index=X.columns
).sort_values(ascending=False)
print("Decision Tree Feature Importance")
print(tree_importance)

tree_importance = pd.Series(
    tree.feature_importances_,
    index=X.columns
).sort_values(ascending=False)
print("Logistic Regression Feature Importance")
print(tree_importance)
```

```
Decision Tree Feature Importance
BMI             1.0
Age             0.0
BloodPressure   0.0
Cholesterol     0.0
Glucose         0.0
FamilyHistory   0.0
dtype: float64
Logistic Regression Feature Importance
BMI             1.0
Age             0.0
BloodPressure   0.0
Cholesterol     0.0
Glucose         0.0
FamilyHistory   0.0
dtype: float64
```

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

An MDP is represented as:

$MDP=(S,A,P,R,\gamma)$

States

Instead of using raw patient values directly, we can discretize patient health into stages:

State	Description
S0	Low-risk / stable
S1	Elevated risk
S2	High-risk
S3	Poorly controlled
S4	Improved/stable

For example, a state could be constructed from:

```
Glucose category
+ BMI category
+ Blood-pressure category
+ diabetes-risk status
```

A real implementation should define these categories with clinical experts and validated thresholds rather than arbitrary cutoffs.

Actions

A={Diet, Exercise, Medication, Monitor}

Meaning

Diet: recommend a clinically approved dietary intervention.

Exercise: recommend an appropriate physical-activity intervention.

Medication: in a real deployment, this must represent a clinician-approved medication-management pathway rather than autonomous prescribing.

Monitor: recommend continued monitoring/reassessment.

Reward Function

A simple educational simulation can use:

Outcome	Reward
Significant health improvement	+10
Small improvement	+5
No change	−1
Health deterioration	−5

For example:

$$R(s,a,s') = \begin{cases} +10 & \text{major improvement} \\ +5 & \text{small improvement} \\ -1 & \text{no improvement} \\ -5 & \text{deterioration} \end{cases}$$

Why?

The agent should learn actions that improve the patient's simulated health state over time rather than maximizing immediate reward only.

Transition Dynamics

For example:

```
High Risk + Diet
       ↓
Elevated Risk

High Risk + Exercise
       ↓
Elevated Risk / Stable

High Risk + Monitor
       ↓
High Risk

Poor Control + Appropriate clinician-managed intervention
       ↓
Improved / Stable
```

In a genuine medical RL system, these transition probabilities should come from validated longitudinal data and clinical knowledge—not invented probabilities.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

Q - Learning

```python
import numpy as np

# States
states = [
    "Low Risk",
    "Elevated Risk",
    "High Risk",
    "Poor Control",
    "Improved"
]

# Actions
actions = [
    "Diet",
    "Exercise",
    "Medication",
    "Monitor"
]

n_states = len(states)
n_actions = len(actions)

# Q-table
Q = np.zeros((n_states, n_actions))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

# Example educational transition environment
# These transitions are illustrative only.
```

```python
# These transitions are illustrative only.
transition_table = {
    (0, 0): (0, 5),    # Low Risk + Diet
    (0, 1): (0, 5),    # Low Risk + Exercise
    (0, 2): (0, 0),    # Low Risk + Medication
    (0, 3): (0, 2),    # Low Risk + Monitor

    (1, 0): (0, 5),    # Elevated + Diet
    (1, 1): (0, 5),    # Elevated + Exercise
    (1, 2): (0, 4),    # Elevated + Medication
    (1, 3): (1, 1),    # Elevated + Monitor

    (2, 0): (1, 5),    # High Risk + Diet
    (2, 1): (1, 5),    # High Risk + Exercise
    (2, 2): (3, 4),    # High Risk + Medication
    (2, 3): (2, -1),   # High Risk + Monitor

    (3, 0): (2, 5),
    (3, 1): (2, 5),
    (3, 2): (4, 10),
    (3, 3): (3, -1),

    (4, 0): (4, 2),
    (4, 1): (4, 2),
    (4, 2): (4, 0),
    (4, 3): (4, 5)
}

# ------------------------------------------------------
# Q-learning
# ------------------------------------------------------

for episode in range(5):

    state = 2    # Start from High Risk
```

```
[32]
✓ 0s
    for iteration in range(10):

        # Epsilon-greedy action
        if np.random.random() < epsilon:
            action = np.random.randint(n_actions)
        else:
            action = np.argmax(Q[state])

        # Environment transition
        next_state, reward = transition_table[
            (state, action)
        ]

        # Q-learning update
        old_q = Q[state, action]

        target = reward + gamma * np.max(Q[next_state])

        Q[state, action] = (
            old_q +
            alpha * (target - old_q)
        )

        state = next_state

print("Final Q-table:")
print(np.round(Q, 3))

...  Final Q-table:
[[14.113  2.442  1.358  0.887]
 [ 0.5    1.535  2.576  0.16 ]
 [ 2.486  0.     0.     0.   ]
 [ 0.     0.     0.     0.   ]
 [ 0.     0.     0.     0.   ]]
```

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Final Learned Policy

```
[33]
✓ 0s
policy = {}

for state in range(n_states):

    best_action = np.argmax(Q[state])

    policy[states[state]] = actions[best_action]

print("\nFinal Learned Policy:")

for state, action in policy.items():
    print(f"{state:20s} -> {action}")

...
Final Learned Policy:
Low Risk             -> Diet
Elevated Risk        -> Medication
High Risk            -> Diet
Poor Control         -> Diet
Improved             -> Diet
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation